
🧠 Queue in Java (Using Collection Framework)

🔷 What is a Queue?

- A **Queue** is a **linear data structure** that follows the **FIFO** (First In, First Out) principle.
- Think of a queue like a line at a ticket counter — first person in is the first one served.

✅ Interface Used: `Queue<E>`

- Belongs to `java.util` package.
- **Extended by**: `Deque<E>` (for double-ended queues).
- **Implemented by**:
    - `LinkedList` ✅ (most commonly used)
    - `PriorityQueue`
    - `ArrayDeque` (for faster, non-thread-safe access)

Method	Description
`add(E e)`	**Inserts element. Throws exception if full.**
`offer(E e)`	**Inserts element. Returns `false` if full (safer).**
`remove()`	**Removes and returns the head. Exception if empty.**
`poll()`	**Removes and returns head. Returns `null` if empty.**
`element()`	**Returns head without removing. Exception if empty.**
`peek()`	**Returns head without removing. Returns `null` if empty.**

⚙️ Queue Methods You Should Know

```
import java.util.*;

public class QueueExample {
    public static void main(String[] args) {
        Queue<Integer> q = new LinkedList<>();

        // Add elements to the queue
        q.add(10);
        q.offer(20);
        q.add(30);

        System.out.println("Queue: " + q);  // [10, 20, 30]

        // Access head element
        System.out.println("Head using peek(): " + q.peek());     // 10
        System.out.println("Head using element(): " + q.element());// 10

        // Remove elements
        System.out.println("Removed using remove(): " + q.remove()); // 10
        System.out.println("Removed using poll(): " + q.poll());     // 20

        System.out.println("Queue after removals: " + q); // [30]
    }
}
```

Find Max in Subarray Using Collections

🔸 **Goal:**

Find the **maximum element** in a specific part (range) of an array using Java Collection classes.

✅ Steps:

1. Convert the array to a List using `Arrays.asList()`.
2. Use `.subList(start, end + 1)` to extract the required range.
3. Use `Collections.max()` to get the maximum value.

```
import java.util.*;

public class MaxInSubArray {
    public static void main(String[] args) {
        Integer[] arr = {5, 8, 3, 12, 7, 9, 15, 2};

        int start = 2;  // starting index
        int end = 6;    // ending index (inclusive)

        List<Integer> list = Arrays.asList(arr);
        List<Integer> subList = list.subList(start, end + 1);

        int max = Collections.max(subList);

        System.out.println("Max value between index " + start + " and " + end + " is: " + max);
    }
}
```

Sliding Window Maximum

Problem Description:

Given an array of integers `arr` and an integer `k`, you need to find the maximum value within each sliding window of size `k` as it moves from the leftmost to the rightmost end of the array.

A sliding window is a contiguous sub-array of fixed size `k` that "slides" one position to the right at a time, revealing a new sub-array. For each position of the window, you must identify and record the largest element present in that specific window.

Input:

- `arr`: An array of integers.
- `k`: An integer representing the size of the sliding window. `1 <= k <= arr.length`.

Output:

- An array of integers, where each element is the maximum value in the corresponding sliding window.

Example:

- **Input:** `arr = [1, 3, -1, -3, 5, 3, 6, 7]`, `k = 3`
- **Output:** `[3, 3, 5, 5, 6, 7]`

Double Ended Queue (Deque)

Problem Description:

Implement a Double Ended Queue (Deque), which is a linear data structure that supports element insertion and deletion from both ends. Unlike a traditional queue (FIFO) or stack (LIFO), a Deque offers the flexibility of adding or removing elements from either the front or the back.

The goal is to create a robust Deque implementation that provides efficient execution for its core operations.

Core Operations to Implement:

1. `addFront(element)`: Adds an `element` to the front of the deque.
2. `addBack(element)`: Adds an `element` to the back of the deque.
3. `removeFront()`: Removes and returns the element from the front of the deque. If the deque is empty, handle this appropriately (e.g., return `null`, `undefined`, or throw an error).
4. `removeBack()`: Removes and returns the element from the back of the deque. If the deque is empty, handle this appropriately.
5. `peekFront()`: Returns the element at the front of the deque without removing it. If the deque is empty, handle this appropriately.
6. `peekBack()`: Returns the element at the back of the deque without removing it. If the deque is empty, handle this appropriately.
7. `isEmpty()`: Returns a boolean indicating whether the deque is empty.
8. `size()`: Returns the number of elements currently in the deque.

Category	Method	Description	Example
Add Elements	`add(E e)`	Adds element at the end	`list.add(10);`
	`add(int index, E e)`	Inserts element at given index	`list.add(1, 20);`
Remove Elements	`remove(int index)`	Removes element at index	`list.remove(0);`

	`remove(Object o)`	Removes first occurrence of object	`list.remove(Integer.valueOf(20));`
	`clear()`	Removes all elements	`list.clear();`
Access/Update	`get(int index)`	Returns element at index	`list.get(1);`
	`set(int index, E e)`	Replaces element at index	`list.set(0, 99);`
Search/Check	`contains(Object o)`	Checks if element exists	`list.contains(15);`
	`indexOf(Object o)`	First index of element	`list.indexOf(10);`
	`lastIndexOf(Object o)`	Last index of element	`list.lastIndexOf(10);`
	`isEmpty()`	Checks if list is empty	`list.isEmpty();`
	`size()`	Returns number of elements	`list.size();`
Utilities	`toArray()`	Converts to Object array	`list.toArray();`
	`toArray(new T[0])`	Converts to specific array type	`list.toArray(new Integer[0]);`
Collections	`Collections.sort(list)`	Sorts the list in ascending order	`Collections.sort(list);`
	`Collections.reverse(list)`	Reverses the list	`Collections.reverse(list);`

`Design Hit Counter

Problem Description:

Design a hit counter that can record hits and return the total number of hits in the past 5 minutes.

Your counter should support two primary operations:

1. **`hit(timestamp)`: Records a hit at the given `timestamp`.**
2. **`getHits(timestamp)`: Returns the number of hits that have occurred in the past 5 minutes (300 seconds) up to the given `timestamp`.**

Assume that `timestamp`s are in seconds and are always increasing. You do not need to consider any invalid `timestamp` values or `timestamp`s that are out of order.

Input:

- **`hit(timestamp)`: An integer `timestamp` representing the time a hit occurred.**
- **`getHits(timestamp)`: An integer `timestamp` representing the current time for which the count of hits in the past 5 minutes is requested.**

Output:

- **`getHits(timestamp)`: An integer representing the total number of hits that occurred within the `[timestamp - 300, timestamp]` interval.**

Example:

Let's illustrate with an example:

HitCounter counter = new HitCounter();

// hit at timestamp 1
counter.hit(1);

// hit at timestamp 2
counter.hit(2);

// hit at timestamp 3
counter.hit(3);

// get hits at timestamp 4
counter.getHits(4); // returns 3 (hits at 1, 2, 3 are all within [1, 4])

// hit at timestamp 300
counter.hit(300);

// get hits at timestamp 300
counter.getHits(300); // returns 4 (hits at 1, 2, 3, 300 are all within [0, 300])

// get hits at timestamp 301
counter.getHits(301); // returns 3 (hit at 1 is now outside the window [2, 301], hits at 2, 3, 300 are within)